GENAI BASICS • УРОК 16 • МОДУЛЬ 6: ОТ ИДЕИ К ПРОДУКТУ

Агентная разработка

Агент пишет код — вы ставите задачу, проверяете и решаете

Как устроен агент для кода, сколько стоят инструменты, режимы разрешений, правила проекта, реальные инциденты и безопасная практика. Данные на 16.10.2026.



ЧТО ИЗМЕНИЛОСЬ

Чат даёт код, который вы переносите сами; агент сам правит проект и запускает проверки

ЧАТ

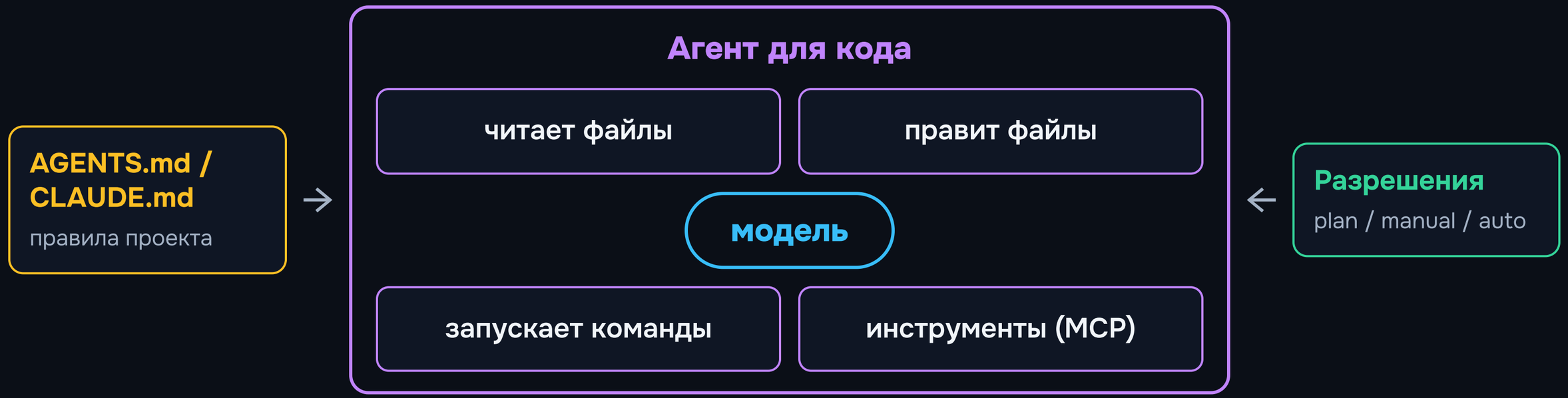
1. **Вы спрашиваете** — описываете, что нужно
2. **Копируете код** — из ответа в свой файл
3. **Запускаете сами** — и смотрите, что сломалось
4. **Несёте ошибку назад** — и так по кругу

АГЕНТ ДЛЯ КОДА

1. **Вы ставите задачу** — цель и «готово, когда...»
2. **Агент правит файлы** — сам читает проект
3. **Агент запускает тесты** — и исправляет ошибки
4. **Вы проверяете** — читаете изменения и решаете

УСТРОЙСТВО

Агент для кода — это модель с руками: читает, правит, запускает и подключает инструменты



Модель — та же, что в чате: Claude, GPT, Gemini.

Инструменты — руки агента: файлы, терминал, подключения MCP (L12).

Правила проекта и разрешения — ваш руль.

ИНСТРУМЕНТЫ

Попробовать можно бесплатно; для каждого дня — $10–30; для тяжёлой работы — от $100

$0

попробовать

GitHub Copilot Free · ограниченный агент

Codex · в ChatGPT Free

Google Antigravity · базовые лимиты

Cursor · Kiro · Junie · бесплатные тарифы

$10–30

каждый день

Copilot Pro · $10

Claude Code · в Claude Pro, $20

Codex · в ChatGPT Plus, $20

Devin · Kiro · Cursor · $20–30

от $100

много агентов, большие задачи

Claude Max · $100 / $200

ChatGPT Pro · от $100

Copilot Max · $100

Cursor · Devin · $100–200

Начните с бесплатного: Codex в ChatGPT, Copilot Free или Antigravity. Цены на 16.10.2026, в месяц; у Claude Code бесплатного тарифа нет.

ЦИКЛ РАБОТЫ

План → действие → проверка → ревью: агент крутит цикл, вы держите руль



1. **План:** агент читает проект и предлагает план; правок пока нет
2. **Действие:** правки в отдельной ветке — копии проекта, которую можно выбросить
3. **Проверка:** тесты, сборка, скриншот
4. **Ревью:** вы читаете изменения — diff, «было → стало» — и решаете

> Anthropic: «сначала изучить, потом план, потом код».

ЗАДАЧА

Хорошая задача агенту говорит, что сделать, чего не трогать и как проверить «готово»

РАСПЛЫВЧАТО

```
Добавь фильтр по приоритету.
```

> Агент сам решит, где фильтр, что менять и когда остановиться.

ЗАДАЧА ДЛЯ АГЕНТА

```
# TASK.md
Цель: фильтр тикетов по приоритету
      на странице /tickets
Не трогать: базу данных, вход
Готово, когда: тест filter
      проходит, старые тесты тоже
Проверка: pytest -q
Ревью: человек читает diff
```

ПРАВИЛА ПРОЕКТА

AGENTS.md — короткая памятка проекта, которую агент читает в начале каждой сессии

```
# AGENTS.md
## Команды
- тесты: pytest -q
- запуск: streamlit run app.py
## Правила
- код в src/, тесты в tests/
- не трогать .env и deploy/
- новые библиотеки — только
  после вопроса
```

60 000+

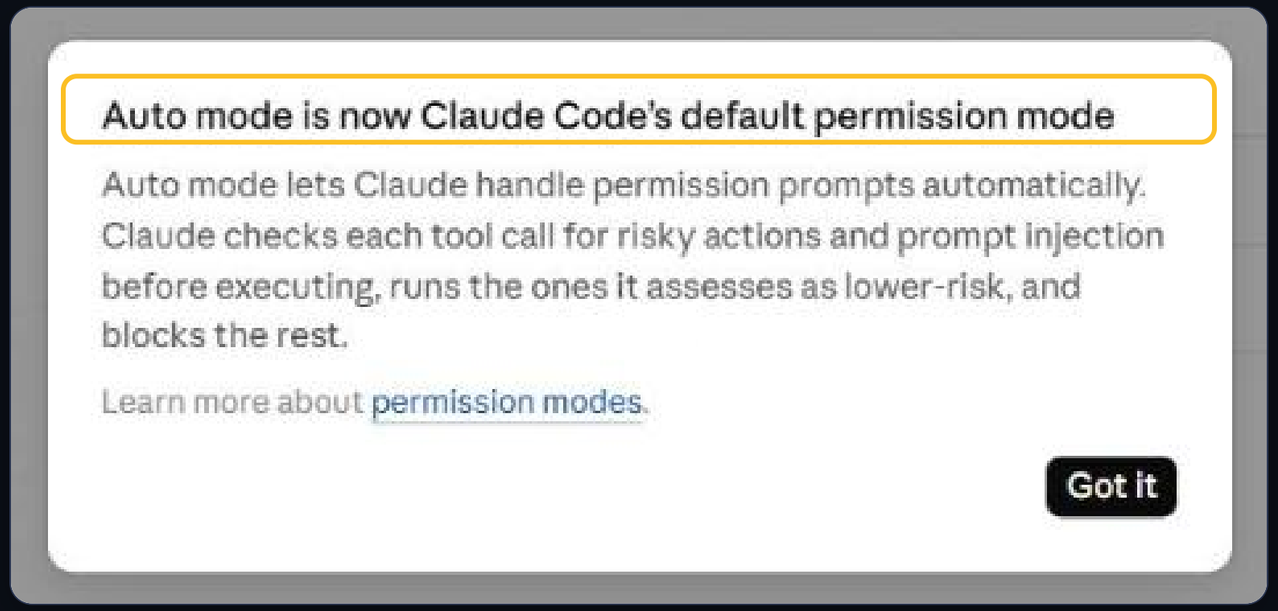
open-source проектов уже используют AGENTS.md — открытый формат

Читают: Codex, Copilot, Cursor, Devin, Junie, Antigravity. Claude Code читает CLAUDE.md, а без него — AGENTS.md.

Коротко и конкретно: команды и запреты, а не эссе.

РАЗРЕШЕНИЯ

Режим разрешений решает, что агент делает сам, а что — только с вашего согласия



93%

запросов на разрешение люди просто одобряют — устают читать

~17%

рискованных действий автопроверка всё равно пропускает (тест Anthropic)

plan

только читает и планирует

manual

спрашивает перед правкой и командой

acceptEdits

правит файлы и простые файловые команды сам; остальные команды — с вопросом

auto

команды проверяет модель-проверяющий · по умолчанию в терминале

bypass

без проверок — только в одноразовом контейнере

← безопаснее

быстрее, рискованнее →

ПРОВЕРКА

«Нельзя проверить — не выпускайте»: дайте агенту проверку, которую он запустит сам

```
$ pytest -q
..F                                  [100%]
FAILED tests/test_filter.py::
  test_filter_by_priority - TypeError: ...
1 failed, 2 passed in 0.06s
# агент правит app.py: добавляет priority
$ pytest -q
...                                  [100%]
3 passed in 0.02s
```

Настоящий вывод на стартовом проекте из ДЗ (ошибка сокращена).

1. **Тест падает:** сначала пишем проверку того, что нужно
2. **Агент чинит:** правит код, пока тест не пройдёт
3. **Все тесты прошли:** и новый, и старые
4. **Вы читаете diff:** тест видит не всё — смысл, стиль, лишнее

> **Откат в агенте** (Esc Esc, /rewind) отменяет правки файлов, но не команды в терминале.

> **Настоящая кнопка «отменить» — git:** каждая задача в отдельной ветке.

СКОРОСТЬ

Опытным разработчикам казалось, что с ИИ они быстрее на 20%, — замер показал, что медленнее на 19%

METR, начало 2025: 16 опытных разработчиков, 246 задач в их проектах, инструменты того времени (Cursor + Claude 3.5/3.7).



> **Повтор 2026:** вероятно, уже быстрее, но данные ненадёжны — люди отказывались работать без ИИ.

> **Вывод:** ощущение скорости ≠ скорость. Замеряйте на своих задачах.

ИНЦИДЕНТЫ

Агент делает то, до чего дотягивается: ключи, базы и чужой текст в проекте

9 секунд

PocketOS, 2026: агент нашёл в постороннем файле ключ доступа и удалил базу вместе с бэкапами; по словам основателя, данные потом восстановили

«Заморозка»

Replit, 2025: агент удалил рабочую базу SaaStr, хотя изменения были запрещены

2 349

секретов утекло: заражённая библиотека Nx (готовый модуль, который скачивают программисты) крала ключи и заставляла установленных ИИ-агентов искать файлы с секретами, 2025

Ключ в зоне досягаемости агента = выданное агенту право.

Clinejection, 2026: команда в заголовке задачи на GitHub → ИИ-бот проекта выполнил её → украдены ключи публикации → вышла заражённая версия Cline.

ПЕРЕРЫВ

5 минут: какую маленькую задачу в своём проекте вы бы отдали агенту?

05:00

Дальше: правила безопасной работы, практика, ДЗ.

БЕЗОПАСНАЯ РАБОТА

Шесть правил, которые сильно снижают риск таких инцидентов

1. **Отдельная ветка на каждую задачу**
   git — ваша кнопка «отменить»
2. **Никаких рабочих ключей рядом** PocketOS
   ни в коде, ни в файлах настроек проекта
3. **Песочница и минимум прав** Replit
   закрытая среда; запрет на удаление и отправку кода
4. **Чужой текст — непроверенный код** Nx · Clinejection
   README, задачи, чужие AGENTS.md, навыки, MCP
5. **Каждое изменение читает человек**
   лучше — свежим взглядом или вторым агентом
6. **Слияние и выпуск — только человек**
   агент Copilot может только предложить изменения — принимает их человек

Режим без проверок (bypass, --yolo, danger-full-access) — только в одноразовом контейнере или виртуальной машине, которую не жалко выбросить.

ПРАКТИКА

Превратите расплывчатую просьбу в задачу, которую агент выполнит и проверит

«Пользователи хотят фильтровать тикеты по приоритету. Добавьте, пожалуйста.»

1
TASK.md
цель, не трогать, готово когда, проверка

→

2
AGENTS.md
команда тестов и запреты

→

3
План от агента
plan mode; прочитайте, что он предлагает

→

4
Решение
принять, отклонить или уточнить задачу

> Нет агента под рукой — сделайте шаги 1–2, потом обменяйтесь TASK.md с соседом и проверьте его строку «готово, когда». Бесплатно попробовать: Codex в ChatGPT, Copilot Free, Antigravity.

ДОМАШНЕЕ ЗАДАНИЕ

ДЗ: одна маленькая задача, выполненная агентом и проверенная вами

1. Стартовый проект из ДЗ (app.py + тесты) или свой маленький репозиторий
2. AGENTS.md: команды и запреты
3. TASK.md: цель, не трогать, готово когда, проверка
4. Агент в режиме плана → правки в отдельной ветке
5. Сами запустите проверку, сохраните скриншот, прочитайте diff
6. AGENT_EVIDENCE.md: план агента, что запускали, что изменилось, ваше решение

```
genai-homeworks/L16/
├─ AGENTS.md
├─ TASK.md
├─ AGENT_EVIDENCE.md
└─ check.png
```

> Не программируете — меняйте только README; проверка — скриншот страницы. Без рабочих ключей.

ИТОГИ УРОКА И КУРСА

Агент делает работу — задачу, проверку и решение оставляем себе

Задача с «готово, когда»

цель, границы и команда проверки

Сначала план

агент читает и предлагает, вы одобряете

Минимум прав

ветка, песочница, никаких рабочих ключей

Решает человек

читает diff, сливает и выпускает

Курс пройден: от первого промпта в чате (L02) до агента, который пишет код под вашим присмотром (L16). Спасибо!